Technical Supplement for GP2PL

Anonymous submission

1 Formal Framework

This appendix makes explicit the mathematical objects, assumptions, and proofs summarized in the main paper. The framework is defined at the level of typed planning transitions (McDermott et al. 1998; Fox and Long 2003) and trigger–context–body BDI rules (Rao 1996; Meneguzzi and De Silva 2015). MOOSE, Clingo, AgentSpeak(L), and Jason instantiate four interfaces; none is built into the semantic definitions below.

1.1 Planning and Evidence

Definition 1 (Typed planning domain)**.** *A domain is $D = \langle \mathcal{P}, \mathcal{F}, \mathcal{A}, \mathcal{T} \rangle$. $\mathcal{P}$ is a finite set of typed predicate schemas, $\mathcal{F}$ a finite set of integer-valued function schemas, $\mathcal{A}$ a finite set of deterministic action schemas, and $\mathcal{T}$ a finite type hierarchy. A problem instance is $I = \langle D, O_I, s_I^0, G_I \rangle$, where O_I is its finite typed object set, s_I^0 its initial state, and G_I its PDDL achievement condition. Write $\mathrm{States}(D, O_I)$ for the typed state set admitted by D over O_I. A state $s = (s_{\mathcal{P}}, s_{\mathcal{F}})$ is a Boolean valuation of ground predicates and a bounded integer valuation of supported ground functions. An applicable ground action a has successor $\gamma(s, a)$ given by its PDDL add, delete, and constant-integer effects.*

Definition 2 (Singleton-goal evidence)**.** *A raw provider artifact $\mathcal{E}_{\mathrm{raw}}$ is admissible when it can be normalized into a finite evidence program E_0, understood as a finite rule set. A normalized evidence rule is*

$$e = \langle \bar{X}_e, C_e^s, g_e, \bar{a}_e, \nu_e, \pi_e \rangle.$$

$\bar{X}_e$ are typed variables, C_e^s is a partial state condition, g_e is one positive predicate literal or supported integer equality, $\bar{a}_e$ is a finite PDDL action macro, ν_e records numeric conditions and effects, and π_e records provenance. The program E_0 is admissible for D only if all symbols, arities, types, and macro actions map to D, and symbolic execution under C_e^s establishes g_e.

Definition 3 (Canonical lifting)**.** *For an admissible provider-normalized program E_0, canonical lifting is the map $\mathrm{Lift}_D(E_0) = E$ that alpha-renames every provider-local variable to a typed canonical variable. The map preserves repeated-variable sharing within and across a rule's goal, context, and macro actions; constants declared by D remain constants. Variables introduced while instantiating typed action schemas are fresh with respect to the lifted evidence rule. Thus lifting is invariant to provider-local variable names and cannot replace a training object by an unrelated constant. It is a representation-level canonicalization of first-order evidence, not an inference procedure from ground plans.*

MOOSE readable policies (Chen et al. 2026) are the evaluated evidence instantiation. For example, a readable rule may associate the singleton goal $on(X, Y)$ with context $holding(X) \wedge clear(Y)$ and action $stack(X, Y)$. GP2PL does not assume that this evidence already implements $clear(Y)$ or covers every reachable context.

Definition 4 (Producible target universe)**.** *For domain D, let $\mathrm{prod}(D) = \{p \mid p$ occurs in a positive add effect of some $a \in \mathcal{A}\}$. If $\mathrm{goalPred}(E)$ is the set of positive predicate symbols targeted by admissible evidence, then*

$$T_D(E) = \mathrm{goalPred}(E) \cup \mathrm{prod}(D).$$

$T_D(E)$ is the Boolean target universe used for action-schema-derived atomic candidate generation. Predicates with no add or delete effect are static and may occur only as contexts. A delete-only dynamic predicate is not an action-schema-derived positive target. The numeric target set $T_D^{\mathbb{Z}}(E)$ contains the admitted singleton integer equalities in E and does not remove any member of $\mathrm{prod}(D)$.

For target set T, the producer–precondition dependency graph is $\mathcal{G}_D^{\mathrm{prod}}(T) = (V_T, E_T)$. Its vertices are predicate signatures reachable from T; $p \to r$ is in E_T exactly when an action-schema producer for p has a positive dynamic precondition with predicate r. Action-schema regression is admitted only on an acyclic such graph. This is a structural condition over renamed schemas, not a domain label.

Atomic compilation		Temporal composition and execution	
Symbol	Unique semantic role	Symbol	Unique semantic role
D, s, γ	Typed planning domain, world state, and deterministic PDDL successor function.	$\tau_q, \theta_q, \widehat{\tau}_q$	Unbound specification, external binding, and validated bound query.
$I, O_I, \mathcal{I}_{\text{train}}$	Problem instance, its typed object set, and finite training instance set.	ι_q, g_q	Query identifier and generated top-level BDI goal.
$\mathcal{E}_{\text{raw}}, E_0, E, e$	Raw provider artifact, provider-normalized program, canonically lifted finite evidence program, and one evidence rule.	$\mu_q, \Theta_q, \Gamma_q$	Proposition map, parameter type signature, and binding constraints.
$T_D(E), T_D^{\mathbb{Z}}(E)$	Boolean producible target universe and admitted integer-equality targets.	$\mathcal{D}_q, val_q, d_q$	Deterministic finite automaton, PDDL-state valuation, and acceptance distance.
$\text{prod}(D), \text{goalPred}(E)$	Predicate symbols with producers and positive predicate evidence.	$q_s, \chi, P_\chi^+, P_\chi^-$	Source state, transition guard, and its positive and negative occurrences.
$\mathcal{G}_D^{\text{prod}}(T)$	Producer–precondition dependency graph reachable from targets T.	$\mathcal{O}_\chi$	Signed transition obligation induced by χ.
$\mathfrak{G}, \text{Inst}_D$	Domain-independent candidate-construction grammar and lifted typed-schema instantiation.	$\Pi_{\chi,i}, \mathcal{G}_\chi, \prec_\chi$	Preservation portfolio, threat-induced precedence graph, and relation.
$\mathcal{C}_E, \mathcal{C}_D(E), \mathcal{C}_{D,E}$	Validated evidence branches, action-schema-derived branches, and generated union.	$\boldsymbol{\ell}_\chi$	Certified serialization of a signed obligation.
$b, \Sigma_b, \text{Cert}_D(b)$	One candidate branch, its conditional completion summary, and soundness predicate.	$\mathcal{W}_{q_s}, \mathcal{I}_{q_s}$	Waiting self-loop guards and signed source invariants.
$\rho_b, \kappa_{\mathcal{M}}, \mathcal{G}_b^{\text{res}}$	Same-predicate recursive ranking, dependency ranking, and resource-mode graph.	$\rho_{\text{num}}, b_\chi^{\text{joint}}$	Numeric-progress ranking and complete joint guard-establishment branch.
$\text{Closed}_D, \text{covers}_D$	Internal-call closure predicate and certified evidence-coverage relation.	$R_\chi[i,j], c_{q_s,\chi}$	Transition-repair subtree and source-state completion test.
$\mathcal{C}_D^{\text{nr}}(E), \text{realizes}_D$	Nonrecursive schema obligations and certified realization relation.	$\text{Pass}_{q_s,\chi}, \mathcal{R}_{q_s,\chi}$	Complete repair pass and plan set for one source-state obligation.
$\mathcal{C}_{D,E}^{\checkmark}, \Omega_{D,E}, \mathfrak{F}_{D,E}$	Certified set, set-level obligations, and feasible selection family.	$\mathcal{Q}_q, \mathcal{L}_D^{[k]}$	Query-local controller plans and maintained library after k appends.
$\mathcal{M}_D$	Lexicographically optimal feasible atomic module core.	ξ, z_i	Committed PDDL trace and runtime DFA state after observation i.
$N_{\text{rel}}, N_{\text{prep}}, N_C, N_\beta$	Counts of selected relation-ranked recursion, acyclic precondition repair, context literals, and body steps.	$\Phi_{\text{syn}}, \Phi_{\text{bench}}, \Phi_{\text{cert}}$	Declared input syntax, evaluated profile family, and certificate-accepted subset.

Table 1: Canonical GP2PL notation. A symbol is not reused across evidence, candidate generation, selected libraries, temporal queries, and runtime traces.

1.2 Parametric LTLf Syntax and Semantics

Definition 5 (Declared input syntax)**.** *For proposition alphabet AP_q, the admitted formula language Φ_{syn} is generated by*

$$\begin{aligned}\ell &::= a \mid \neg a,\\ \varphi &::= \ell \mid (\varphi \wedge \varphi) \mid F\varphi \mid X\varphi\\ &\quad \mid (\varphi U \varphi), \qquad a \in AP_q.\end{aligned}$$

Negation is restricted to atoms and is written ! in serialized input. The syntax contains no disjunction, global, release, weak-next, implication, equivalence, or quantifier. Parsing a formula in Φ_{syn} does not assert that a PDDL action strategy exists.

Definition 6 (Bound proposition valuation)**.** *For validated bound query $\widehat{\tau}_q = (\tau_q, \theta_q)$, substitution θ_q replaces declared parameters with type-compatible objects and leaves domain constants fixed. Its proposition valuation is*

$$a \in val_q(s) \iff \begin{cases} p(\bar{t})\theta_q \in s_{\mathcal{P}}, & \mu_q(a) = p(\bar{t}),\\ s_{\mathcal{F}}(f(\bar{t})\theta_q) = c, & \mu_q(a) = [f(\bar{t}) = c]. \end{cases}$$

Thus predicate atoms and supported integer equalities are evaluated in the same bound PDDL state.

Definition 7 (Finite-trace satisfaction)**.** *Let $\xi = s_0 \dots s_n$ be a nonempty finite state trace and $0 \le i \le n$. Satisfaction for Φ_{syn} is*

$$\begin{aligned}\xi, i \models a &\iff a \in val_q(s_i),\\ \xi, i \models \neg a &\iff a \notin val_q(s_i),\\ \xi, i \models \varphi \wedge \psi &\iff \xi, i \models \varphi \text{ and } \xi, i \models \psi,\\ \xi, i \models X\varphi &\iff i < n \text{ and } \xi, i+1 \models \varphi,\\ \xi, i \models F\varphi &\iff \exists j \in [i, n] : \xi, j \models \varphi,\\ \xi, i \models \varphi U \psi &\iff \exists j \in [i, n] : \xi, j \models \psi \text{ and}\\ &\qquad \forall k \in [i, j) : \xi, k \models \varphi.\end{aligned}$$

Write $\xi \models \varphi$ for $\xi, 0 \models \varphi$. The clauses make X strong and require the right operand of U to occur on the finite trace.

Definition 8 (Evaluation and certified-compilation scopes)**.** *$\Phi_{\text{bench}} \subset \Phi_{\text{syn}}$ is the family obtained by instantiating the five registered templates in Table 6. For fixed D and $\mathcal{M}_D$, $\Phi_{\text{cert}}(D, \mathcal{M}_D) \subseteq \Phi_{\text{syn}}$ contains validated bound formulas whose required distance-reducing DFA obligations admit the binding, completion-effect, preservation, progress, and resource certificates used by the compiler. Formula parsing alone therefore establishes membership only in Φ_{syn}.*

MONA may encode alternatives in the Boolean transition relation as several valuation cubes. Each cube is compiled as a separate conjunctive guard. Several cubes, including cubes sharing a source and target, are not one disjunctive guard: the runtime monitor evaluates each original cube, while the compiler may share only a certified common achievement objective. An explicit disjunction inside one cube or an uncertified strategy choice remains outside $\Phi_{\text{cert}}(D, \mathcal{M}_D)$.

1.3 Candidate BDI Modules

Definition 9 (Candidate branch)**.** *A lifted candidate branch is $b = \langle g_b, C_b, \beta_b, \Sigma_b, \pi_b \rangle$, where g_b is one achievement literal, C_b is a conjunction of typed positive and negative state literals, integer comparisons, and equality filters, β_b is a finite sequence of primitive PDDL actions and internal achievement calls, Σ_b is a conditional module-completion summary, and π_b records whether the branch came from provider evidence or action-schema construction.*

Definition 10 (Conditional module-completion summary)**.** *For candidate branch b, Σ_b is a condition-indexed conservative summary of effects observable when β_b returns. Its Boolean projections are MustAdd_b, MayAdd_b, and MayDelete_b; it additionally records any exact constant-integer delta and keyed resource-mode transition. A must effect holds for every feasible selected execution under its recorded condition. A may effect contains every feasible selected completion effect and may overapproximate it.*

Definition 11 (Internal-call closure)**.** *For any BDI plan set $\mathcal{R}$, internal-call closure is the predicate*

$$\begin{aligned}\text{Closed}_D(\mathcal{R}) \iff &\forall b \in \mathcal{R},\ \forall !q(\bar{Y}) \text{ occurring in } \beta_b,\\ &\exists b' \in \mathcal{R} \text{ such that}\\ &\quad g_{b'} \text{ type-unifies with } q(\bar{Y})\\ &\quad \text{under the type hierarchy of } D.\end{aligned}$$

The condition applies to every plan in $\mathcal{R}$ and therefore also to calls reached transitively from another module. Primitive PDDL actions are not achievement calls and are outside this predicate. The predicate establishes type-compatible resolution only: it neither quantifies over reachable states nor guarantees that a resolving branch is applicable in every such state.

The context is an applicability condition, not a sequence of subgoals. Positive context literals and static sort facts may bind variables. Negative literals and comparisons only filter already bound variables. A plan such as

$$+!on(X, Y) : \neg clear(Y) \leftarrow !clear(Y); !on(X, Y)$$

is therefore admitted only when the call to $clear(Y)$ is closed by another selected module and the recursive call carries a well-founded progress proof.

Definition 12 (Candidate soundness certificate)**.** *For domain D, $\text{Cert}_D(b)$ holds exactly when every obligation required by the structure of b is established:*

1. *every action and subgoal variable is range restricted and type compatible;*
2. *symbolic progression proves each primitive precondition;*
3. *successful completion establishes the trigger goal g_b;*
4. *every internal call has a declared, type-compatible target signature and records an internal-call obligation for set-level selection;*
5. *every recursion cycle has a non-negative ranking that strictly decreases;*
6. *every temporary resource occupation has a finite, target-preserving discharge path; and*
7. *Σ_b conservatively contains every feasible completion add and delete effect, including effects propagated through internal calls.*

Obligations that do not arise in b, such as resource discharge for a direct producer that acquires no resource, are vacuous rather than unproved.

Branch constructor	Additional acceptance obligation	Excluded failure
Validated evidence macro	every macro step maps to a PDDL action schema and symbolic execution establishes the singleton evidence target	invalid or misbound provider evidence
Direct producer	one action adds the target under the certified context and preserves its completion summary	action chosen only by predicate name
Acyclic regression	each open positive requirement is discharged without a repeated normalized producer role or conflicting delete effect	arbitrary depth cutoffs or cyclic schema search
Relational recursion	a non-negative relation feature strictly decreases and an already-satisfied, direct-producer, or validated-evidence terminal branch remains available	recursive oscillation
Target-preserving resource discharge	a finite non-repeating mode path restores keyed availability while preserving the target	unreleased reusable resource
Ranked precondition repair	Clingo selects an acyclic call graph with strictly decreasing natural-number dependency ranks	mutually recursive preparation without a progress proof

Table 2: Candidate constructors and soundness obligations for atomic BDI modules. Every selected branch additionally satisfies typed binding, symbolic PDDL executability, and target achievement on return; the resulting core satisfies $\mathrm{Closed}_D(\mathcal{M}_D)$.

Finite candidate-construction grammar. Let

$$\mathfrak{G} = \{\text{producer}, \text{regression}, \text{recursion}, \text{precondition repair}, \text{resource discharge}, \text{numeric progress}\}.$$

Let $\mathcal{C}_E$ be validated evidence macros and define

$$\mathcal{C}_D(E) = \mathrm{Inst}_D(\mathfrak{G}, T_D(E), T_D^{\mathbb{Z}}(E)).$$

The instantiated set $\mathcal{C}_D(E)$ contains direct producers, finite backward STRIPS regressions over acyclic producer–precondition dependencies, certified relational recursion, and finite causal resource-mode discharge. The regression construction builds on goal-regression planning (Fikes, Hart, and Nilsson 1972; Chen et al. 2026). A regression state is an alpha-normalized tuple of open requirements, the selected producer role, and resource modes. A path stops before repeating such a state. Equivalent states retain only their shortest body for the same completion summary. This establishes finiteness without an arbitrary action-depth constant. Provider macros are validated but never truncated by this search rule.

Definition 13 (Evidence coverage)**.** *A certified candidate b covers evidence obligation e when their triggers are alpha equivalent and one of the following holds: their bodies and numeric summaries are alpha equivalent; b has the same body under a logically weaker conjunctive context; or b is an action-schema producer refinement with the same* $\mathrm{MustAdd}$*, no additional* $\mathrm{MayDelete}$*, and equivalent numeric and keyed-resource summaries. Prefix similarity alone is not coverage. We denote this relation by* $\mathrm{covers}_D(b, e)$.

Definition 14 (Schema-achievement coverage)**.** *Let $\mathcal{C}_D^{\mathrm{nr}}(E)$ contain the nonrecursive action-schema-derived branches. A certified branch b realizes $c \in \mathcal{C}_D^{\mathrm{nr}}(E)$, written* $\mathrm{realizes}_D(b, c)$*, when either b and c have alpha-equivalent trigger, context, and body; b is an action-schema producer refinement with a weaker context and no weaker completion contract; or b is a target-preserving resource-discharge alternative with the same achievement contract. This relation never licenses body-prefix similarity alone.*

Definition 15 (Feasible selection family)**.** *The generated candidate set and its certified subset are*

$$\mathcal{C}_{D,E} = \mathcal{C}_E \cup \mathcal{C}_D(E), \qquad \mathcal{C}_{D,E}^{\checkmark} = \{b \in \mathcal{C}_{D,E} \mid \mathrm{Cert}_D(b)\}.$$

Let $\Omega_{D,E}$ contain evidence coverage, nonrecursive schema-achievement coverage, preparation acyclicity, and recursive-ranking compatibility. Per-branch binding, execution, achievement, completion-effect, and resource-discharge obligations are already contained in Cert_D*. The feasible selection family is*

$$\mathfrak{F}_{D,E} = \{\mathcal{M} \subseteq \mathcal{C}_{D,E}^{\checkmark} \mid \mathcal{M} \models \Omega_{D,E} \wedge \mathrm{Closed}_D(\mathcal{M})\}.$$

Thus every $\mathcal{M} \in \mathfrak{F}_{D,E}$ covers every evidence rule and required nonrecursive action-schema achievement contract, is internally closed, and admits the selected recursion and cross-module dependency ranks. Equivalently, schema-achievement coverage requires

$$\forall c \in \mathcal{C}_D^{\mathrm{nr}}(E), \quad \exists b \in \mathcal{M} : \mathrm{realizes}_D(b, c).$$

The certified atomic module core is selected directly as

$$\mathcal{M}_D = \operatorname*{lexargmin}_{\mathcal{M} \in \mathfrak{F}_{D,E}} \mathrm{Cost}(\mathcal{M}),$$

where Cost is the lexicographic objective in the main paper. The core is represented by BDI trigger–context–body rules. It is query-independent, closed under its internal achievement calls, and contains no query identifier, DFA state, or monitor symbol. The evaluated AgentSpeak(L) realization preserves these rules but does not define the core.

Definition 16 (Maintained domain library)**.** *For temporal query q, let $\mathcal{Q}_q$ contain its top-level goal, DFA dispatcher, and certified transition-repair plans. For an ordered sequence of appended queries $q_1, \ldots, q_k$, the maintained domain library $\mathcal{L}_D^{[k]}$ is*

$$\mathcal{L}_D^{[0]} = \mathcal{M}_D, \qquad \mathcal{L}_D^{[k]} = \mathcal{M}_D \cup \bigcup_{i=1}^{k} \mathcal{Q}_{q_i}.$$

$\mathcal{M}_D$ and $\mathcal{Q}_{q_i}$ are logical subsets of this one library, not separately maintained plan-library artifacts. The compiler establishes $\mathrm{Closed}_D(\mathcal{L}_D^{[0]})$ *and accepts an append only when* $\mathrm{Closed}_D(\mathcal{L}_D^{[k+1]})$ *holds. Consequently,* $\mathrm{Closed}_D(\mathcal{L}_D^{[k]})$ *is an invariant of every maintained library.*

GP2PL asks a Clingo answer-set optimizer (Gebser et al. 2019) to compute $\mathcal{M}_D$ by minimizing the declared lexicographic cost over $\mathfrak{F}_{D,E}$. The result is globally optimal only over $\mathcal{C}^{\checkmark}_{D,E}$; candidate generation is not claimed complete for arbitrary planning programs.

1.4 Explicit Assumptions and Rejection Boundary

Assumption 1 (Planning fragment)**.** *Boolean actions use deterministic typed STRIPS semantics. Numeric effects are constant integer increases or decreases over finite values. Arbitrary expressions, continuous change, conditional effects outside the supported PDDL fragment, and probabilistic actions are rejected.*

Assumption 2 (Evidence boundary)**.** *Evidence is finite, singleton-goal, and schema mappable. It may be incomplete, but a provider macro accepted as evidence must pass symbolic PDDL execution. Negative achievement evidence without a schema-validated achieving action sequence is rejected.*

Assumption 3 (Recursive progress)**.** *A same-predicate recursive branch is accepted only when an action-schema-derived ranking function $\rho_b : \mathrm{States}(D) \to \mathbb{N}$, obtained from a non-negative relational feature, strictly decreases until a producer or already-satisfied exit applies. An acyclic support-depth orientation additionally assumes that the grounded support relation remains acyclic in reachable states. Cross-predicate precondition repair uses a selected acyclic call graph for a candidate core $\mathcal{M}$ and a dependency rank $\kappa_{\mathcal{M}}$ over its trigger signatures such that $\kappa_{\mathcal{M}}(g) > \kappa_{\mathcal{M}}(h)$ for every selected call edge $g \to h$.*

Assumption 4 (Target-preserving resource discharge)**.** *A reusable-resource sequence is accepted only when effect unification identifies a stable capacity key. Its abstract resource-mode graph is $\mathcal{G}_b^{\mathrm{res}} = (V_b^{\mathrm{res}}, E_b^{\mathrm{res}})$, where a vertex is an alpha-normalized keyed occupancy mode and a labelled edge $(u, a, v) \in E_b^{\mathrm{res}}$ denotes a symbolically executable action-schema transition from mode u to mode v that preserves the atomic target. The graph must be finite, and symbolic execution proves that the discharge removes occupation, restores availability, and preserves the atomic target. Ambiguous capacity roles are rejected.*

Assumption 5 (Temporal fragment)**.** *The input formula belongs to Φ_{syn} and is translated by the LTLf2DFA/MONA construction (Fuggitti 2020; Henriksen et al. 1995). The evaluation distribution is Φ_{bench}; sound temporal compilation additionally requires membership in $\Phi_{\mathrm{cert}}(D, \mathcal{M}_D)$. A chosen progress cube is a conjunction of signed predicate literals and supported integer equalities. Explicit disjunction requiring strategy choice and unsupported arithmetic are rejected.*

Assumption 6 (Primitive-step observation)**.** *The deterministic finite automaton consumes the initial valuation and every state produced by a successful primitive PDDL action. Only the environment can update query-local monitor beliefs. Atomic module return is not the semantic observation boundary of the full method.*

Construct	Accepted strategy	Rejection boundary
Positive predicate	Certified atomic module	No executable achieving branch
Numeric equality	Bounded constant-integer progress	Unsupported arithmetic or incomplete numeric effect
Positive conjunction	Threat-safe serialization	Incomplete effects or cyclic threats
Negative predicate	Observe absence or use an exact certified deleter	Synthetic negated achievement or possible re-addition
Mixed Boolean and numeric guard	Complete net-effect and preservation certificate	Incomplete mixed-effect certificate
Boolean alternatives	Separate conjunctive MONA cubes	Explicit disjunction in the input or one cube; uncertified strategy choice
Primitive-state safety	DFA monitor after every primitive action	Completion-only observation

Table 3: Supported compilation fragment and fail-closed boundary. Rejection means that GP2PL emits a structured diagnostic rather than an uncertified controller or synthetic atomic goal.

2 Complete Proofs

2.1 Atomic Compilation

Lemma 1 (Range-restricted grounding)**.** *If the binding obligation for branch b holds and its context is true under a type-consistent grounding θ, then every variable in every primitive action and internal call of β is grounded by θ or by a preceding positive context witness.*

Proof. The certificate constructs the bound-variable set from trigger variables, positive context literals, and certified static sort witnesses. A body term is accepted only if it is a constant or belongs to this set. Negative contexts and comparisons are checked after their arguments enter the set and never add a binding. Induction over body positions therefore yields a ground, type-compatible term tuple at every primitive action or call. □

Lemma 2 (Symbolic progression soundness)**.** *Let $\beta = a_1; \ldots; a_m$ be an action-only body certified executable from context C. For every ground state $s \models C$ and compatible grounding θ, each $a_i\theta$ is applicable in the state reached after the prefix, and the symbolic final facts and numeric values hold in the concrete successor.*

Proof. The symbolic state initially contains every positive fact and numeric equality required by $C\theta$. At step i, certification requires every positive, negative, equality, and numeric precondition of $a_i\theta$ in that symbolic state. The update

removes delete effects, applies add effects, and applies each constant numeric delta, exactly matching γ. Induction on i proves that the concrete and symbolic states agree on every tracked requirement. The final symbolic requirements therefore hold concretely. □

Lemma 3 (Well-founded recursive calls)**.** *A selected recursive component satisfying either a strictly decreasing nonnegative relational ranking or strictly decreasing natural-number module ranks cannot execute infinitely while remaining inside that component.*

Proof. For same-predicate recursion, every recursive return-to-trigger step decreases ρ_b and no selected body can restore the protected portion of the ranked relation. An infinite descent in $\mathbb{N}$ is impossible. For cross-predicate precondition repair, every call edge decreases $\kappa_{\mathcal{M}}$, so a call cycle would imply $\kappa_{\mathcal{M}}(g_0) > \kappa_{\mathcal{M}}(g_1) > \cdots > \kappa_{\mathcal{M}}(g_0)$, a contradiction. Each terminal rank has a direct producer, provider macro, or already-satisfied exit because the corresponding nonrecursive schema-achievement contract is covered. □

Lemma 4 (Finite target-preserving resource discharge)**.** *If a resource-discharge certificate exists for a producer body, its discharge sequence terminates, restores the keyed availability mode, removes the acquired occupation mode, and leaves the atomic target true.*

Proof. The abstract search state contains the capacity key and normalized occupation mode. The accepted path does not repeat a mode, so finiteness of the mode graph implies termination. Each edge is a PDDL action validated by symbolic progression. Acceptance explicitly requires deletion of the current occupation, restoration of the matched availability fluent, and absence of a delete effect on the established target. Lemma 2 transfers these properties to concrete execution. □

Lemma 5 (Conservative completion summary)**.** *For a selected closed module call, every Boolean completion effect possible under a feasible selected branch is contained in its conditional* MayAdd *or* $\mathrm{MayDelete}$ *summary, and every* $\mathrm{MustAdd}$ *effect holds for all feasible selected branches under the recorded condition.*

Proof. Primitive branch summaries are computed by exact sequential add/delete progression, so internally deleted-and-restored facts are not reported as final deletes. For each recorded branch condition, the selected program contains a finite universe of alpha-normalized effect templates. A worklist initializes each summary with its primitive net effects, then substitutes the current summary of every selected internal call.

Each update can only add a feasible template to a may set or remove a template from a must set when a feasible alternative does not establish it. These two directions are monotone and the template universe is finite, so the worklist terminates. Ranked recursion is propagated at the same trigger anchors; cross-module calls follow their finite dependency ranks.

Induction on a finite terminating call unfolding shows that every completion effect of depth k appears after at most k propagation rounds. Lemma 3 makes every selected unfolding finite. At convergence, union over feasible alternatives therefore contains every may effect, while intersection over compatible alternatives retains only must effects common to all of them. □

Proposition 1 (Atomic branch soundness)**.** *If an emitted branch for $p(\bar{X})$ has a true context under a type-consistent grounding, and each called selected module satisfies its certified achievement and completion summary, then the branch is executable and successful return establishes $p(\bar{X})$.*

Proof. Lemma 1 grounds every body element. Consider body positions in order. Primitive actions are executable and preserve the certified symbolic state by Lemma 2. For an internal call, the induction hypothesis on the internally closed selected module establishes its goal and its conservative completion summary updates the caller's symbolic state; Lemma 5 ensures that this update omits no feasible completion effect. Lemma 3 excludes infinite selected recursion, and Lemma 4 completes every certified target-preserving keyed resource-mode discharge. The branch achievement obligation requires the grounded trigger in the final symbolic state; Lemma 2 and the call induction therefore establish it in the concrete return state. □

Lemma 6 (Answer-set encoding correspondence)**.** *For every $\mathcal{M} \subseteq \mathcal{C}^{\checkmark}_{D,E}$, the grounded selection program has an answer set whose selected candidates are exactly $\mathcal{M}$ if and only if $\mathcal{M} \in \mathfrak{F}_{D,E}$. For every such answer set, its prioritized optimization vector equals* $\mathrm{Cost}(\mathcal{M})$.

Proof. The program contains one choice atom for each certified candidate. Its coverage rules derive a witness exactly for the relations covers_D and $\mathrm{realizes}_D$ encoded in the finite instance. Integrity constraints reject a selection precisely when an evidence obligation, a nonrecursive schema-achievement obligation, or an internal call lacks a witness.

Additional rank atoms witness the selected acyclic preparation graph and the declared recursive compatibility conditions. Thus projecting any answer set to its choice atoms yields a member of $\mathfrak{F}_{D,E}$. Conversely, a feasible set supplies every required coverage and rank witness, so extending its choice atoms with those derived witnesses satisfies every rule and constraint.

Finally, each weak-constraint term is the corresponding component of the declared lexicographic cost, with the same priority order. Projection therefore preserves both feasibility and objective value. □

Proposition 2 (Certified-candidate-set optimality)**.** *If the answer-set optimizer returns $\mathcal{M}_D$, then $\mathcal{M}_D$ satisfies every encoded evidence-coverage, schema-achievement, internal-call, preparation-acyclicity, and recursive-ranking obligation and is lexicographically optimal among feasible subsets of $\mathcal{C}^{\checkmark}_{D,E}$.*

Proof. By Lemma 6, answer sets correspond exactly to $\mathfrak{F}_{D,E}$ and carry the same cost vector. Clingo's prioritized optimization returns an answer set minimal in lexicographic priority order. Its projected selection is therefore feasible and no other

member of $\mathfrak{F}_{D,E}$ has lower cost. The correspondence concerns only the generated certified candidates and says nothing about ungenerated programs. □

2.2 Temporal Composition

An unbound temporal specification is $\tau_q = \langle \iota_q, \varphi_q, \mu_q, \Theta_q, \Gamma_q \rangle$: identifier, LTL_f formula, proposition-to-PDDL atom map, parameter type signature, and binding constraints. Formally, $\Theta_q : \bar{X}_q \to \mathcal{T}$ is the type signature, and a runtime binding $\theta_q : \bar{X}_q \to O_I$ satisfies Θ_q and Γ_q, and $\widehat{\tau}_q = (\tau_q, \theta_q)$ is the validated bound query. Let

$$\mathcal{D}_q = \langle Q_q^{\mathrm{dfa}}, 2^{AP_q}, \delta_q, q_q^0, F_q \rangle$$

be its deterministic finite automaton obtained from the MONA-backed translation (De Giacomo and Vardi 2013; Fuggitti 2020). For transition guard χ, let

$$\chi = \bigwedge_{G \in P_\chi^+} G \wedge \bigwedge_{N \in P_\chi^-} \neg N, \qquad \mathcal{O}_\chi = \langle P_\chi^+, P_\chi^- \rangle.$$

$\mathcal{O}_\chi$ is its signed transition obligation. A preservation portfolio $\Pi_{\chi,i} \subseteq \mathcal{M}_D$ assigns one or more certified branches to positive occurrence $G_i \in P_\chi^+$. A negative literal is either observed absent or has an exact certified deleter; it never invokes a synthetic negative goal. The conditional definite/potential module-completion summaries follow the goal- interference distinction used in BDI plan analysis (Thangarajah, Padgham, and Winikoff 2003); GP2PL derives them by symbolic PDDL execution and propagates them through selected internal calls.

In $\mathcal{D}_q$, Q_q^{dfa} is the finite state set, AP_q is the proposition alphabet, δ_q is the deterministic transition function, q_q^0 is the initial state, and F_q is the accepting set. The map $val_q : \mathrm{States}(D) \to 2^{AP_q}$ gives the valuation induced by a PDDL state. Define the acceptance distance $d_q(z)$ as the shortest directed-path length from $z \in Q_q^{\mathrm{dfa}}$ to F_q, and $d_q(z) = \infty$ when no such path exists. A certified progress edge (q_s, χ, q_t) must satisfy $d_q(q_t) < d_q(q_s) < \infty$.

Definition 17 (Threat-induced precedence)**.** *For positive occurrences G_i, G_j, define $G_j \prec_\chi G_i$ exactly when a feasible conditional* MayDelete *effect of a branch in $\Pi_{\chi,j}$ unifies with G_i. The threat-induced precedence graph is $\mathcal{G}_\chi = (P_\chi^+, \prec_\chi)$. The ordering constraint requires G_j before G_i. A positive repair is inadmissible when a feasible* MayAdd *effect unifies with a protected $N \in P_\chi^-$.*

The certified serialization $\boldsymbol{\ell}_\chi$ is rendered as a balanced binary transition-repair tree. A positive leaf observes or achieves its atom; a negative leaf observes absence or invokes its exact certified deleter. The tree indexes these occurrences, not DFA states or primitive actions. A direct linear rendering would execute the same certified order but place all n literal calls and the completion test in one monolithic plan body, so the representation parsed and stored for that plan, together with the intention continuation constructed and resumed by the interpreter, would grow with the guard. Midpoint decomposition preserves the identical left-to-right order while limiting every generated controller plan to at most two body steps. Algorithm 1 constructs one such transition plan set.

Algorithm 1 Compile a Certified DFA Progress Controller

Require: DFA source q_s, guard χ, signed obligation $\mathcal{O}_\chi$, selected atomic core $\mathcal{M}_D$
Ensure: Transition-repair plan set $\mathcal{R}_{q_s,\chi}$ or rejection

$\boldsymbol{\Sigma} \leftarrow \{\Sigma_b \mid b \in \mathcal{M}_D\}$
$(\boldsymbol{\ell}_\chi, \boldsymbol{\Pi}_\chi) \leftarrow$ CertifySerialization$(\mathcal{O}_\chi, \boldsymbol{\Sigma})$
if $(\boldsymbol{\ell}_\chi, \boldsymbol{\Pi}_\chi) = \bot$ **then**
  reject
end if
emit entry $r_{q_s,\chi} \leftarrow !R_\chi[1,n]; !c_{q_s,\chi}$
procedure Build(i, j)
if $i = j$ **then**
  emit the satisfied plan for signed literal ℓ_i
  if $\Pi_{\chi,i} \neq \varnothing$ **then**
    emit the unmet plan restricted to $\Pi_{\chi,i}$
  end if
else
  $m \leftarrow \lfloor (i+j)/2 \rfloor$
  emit $R_\chi[i,j] \leftarrow !R_\chi[i,m]; !R_\chi[m+1,j]$
  Build(i,m); Build$(m+1,j)$
end if
end procedure; Build$(1,n)$
emit completion test $c_{q_s,\chi}$: return if monitor $\neq q_s$; otherwise call $!r_{q_s,\chi}$
return $\mathcal{R}_{q_s,\chi}$

Suppose the monitor leaves q_s while a branch in $\Pi_{\chi,i}$ executes. After that call returns, every suffix leaf either observes its literal without acting or selects only a branch from its already certified $\Pi_{\chi,j}$. Applicability is rechecked at invocation; an applicable call is PDDL-executable and preserves the established signed prefix at module return, while a missing call fails without inserting an uncertified action. Every primitive suffix action is observed by the same DFA, so it may traverse further actual edges, including a rejecting edge, but cannot manufacture a transition or report false acceptance. Only after the suffix finishes does the completion test return to top-level dispatch at the current state or replay when that state is still q_s. For $n = 1$, $R_\chi[1,1]$ is one leaf followed by the same completion test; balancing does not choose the certified order or reduce primitive action count, but bounds trigger fan-out and individual plan-body length by two, with logarithmic nesting depth and linear work per pass.

Definition 18 (Complete transition-repair pass)**.** *For a certified serialization $\boldsymbol{\ell}_\chi = (\ell_1, \ldots, \ell_n)$, let*

$$\begin{aligned} R_\chi[i,i] &= \mathrm{Leaf}(\ell_i, \Pi_{\chi,i}), \\ R_\chi[i,j] &= R_\chi[i,m]; R_\chi[m+1,j] \quad (i < j), \\ m &= \lfloor (i+j)/2 \rfloor. \end{aligned}$$

Let $c_{q_s,\chi}$ be the source-state completion test. The complete pass is

$$\mathrm{Pass}_{q_s,\chi} := R_\chi[1,n] \cdot c_{q_s,\chi},$$

where $\cdot$ denotes sequential controller composition. Each leaf observes its signed literal or invokes its optional certified repair call. The runtime monitor advances at every primitive action, but the completion test checks whether the monitor still equals the source state only after $R_\chi[1,n]$ returns.

Lemma 7 (Signed serialization invariant)**.** *Suppose ℓ_χ is a topological order of the threat-induced precedence relation, or an order equipped with per-occurrence preserving portfolios. After the repair of prefix $\ell_1, \ldots, \ell_i$ returns, every positive literal in the prefix holds and every protected negative literal is absent.*

Proof. The base case follows from observing or establishing ℓ_1. Assume the invariant for $i-1$. If ℓ_i is positive, its selected branch establishes it by Proposition 1. Every feasible completion delete that could affect an earlier positive would induce a threat edge and hence contradict the order, unless the per-occurrence portfolio explicitly excludes that branch. Every feasible add of a protected negative is likewise excluded. If ℓ_i is negative, the leaf either observes absence or uses an exact finite deleter whose complete net effects preserve the prefix. Thus the invariant holds by induction. □

Lemma 8 (Post-exit suffix safety)**.** *Suppose the monitor leaves source state q_s during the repair call for ℓ_i. After that call returns, the remaining suffix of the complete transition-repair pass can commit only PDDL-executable actions from certified repairs, preserves the signed serialization prefix at module-return boundaries, and induces only transitions of the runtime DFA. An unavailable repair, illegal primitive action, or rejecting monitor state cannot be converted into controller success.*

Proof. Proceed over suffix occurrences $\ell_{i+1}, \ldots, \ell_n$. A satisfied leaf commits no action. An unmet leaf either has no repair and fails closed, or selects an applicable certified repair. Atomic branch soundness gives PDDL executability and establishment of that occurrence; the signed-serialization invariant excludes completion effects that violate the established prefix. This argument depends on the branch context and symbolic effects, not on the monitor remaining at q_s. By the primitive-step observation assumption, every committed primitive action updates the DFA from its actual current state, and no controller plan can write that state. The suffix can therefore traverse further real edges, but no hidden or fabricated edge. A rejecting state lacks an accepting top-level plan, so any such trace fails rather than being relabelled successful. □

Proposition 3 (Supported transition-composition soundness)**.** *For an accepted signed transition obligation $\mathcal{O}_\chi$, assume every required repair has an applicable certified branch, every selected call terminates, and the preservation portfolios satisfy the signed serialization invariant. At the end of a complete transition-repair pass, the controller retries if the monitor reports its source state and otherwise returns to top-level dispatch at the current state. Every monitor transition during the pass is an actual DFA edge whose complete guard is true in an observed state; a rejecting state cannot report success.*

Proof. The signed-serialization lemma establishes every repaired prefix, independently of whether the primitive-step monitor remains at the source. If it leaves during a repair, the post-exit suffix lemma covers the rest of the complete transition-repair pass. After every successful primitive action, the monitor evaluates the original complete outgoing cubes, not the serialized approximation. When $R_\chi[1, n]$ returns, the completion test reads the current state: it restarts the same obligation exactly when that state is the source and otherwise returns to top-level dispatch. A macro and the remaining suffix may cross several edges, but each is an actual deterministic transition and a rejecting state has no accepting shortcut. No controller statement can manufacture a state transition. □

Proposition 4 (Balanced transition-repair complexity)**.** *For $n \geq 1$ signed literals and e certified repair plans, midpoint splitting produces $2n + e + 2$ query-local plans, trigger fan-out at most two, at most two body steps per generated controller plan, nesting depth $O(\log n)$, and $O(n)$ node visits per pass while preserving the certified literal order.*

Proof. There are n satisfied-leaf plans, e repair alternatives, $n-1$ internal binary nodes, one transition-entry plan, and two completion-test plans, totaling $n+e+(n-1)+1+2 = 2n+e+2$. Each internal trigger has one dispatch plan and each leaf has at most the satisfied and repair alternatives counted above, so structural dispatch fan-out is two. Midpoint division yields height $\lceil \log_2 n \rceil$ up to constant entry and completion frames. Induction on a contiguous range shows that its left subtree completes before its right subtree, so an in-order traversal equals the certified sequence. Each node is visited once per pass, hence linear work. Entry and internal plans contain two controller calls, a repair leaf contains at most its certified call and monitor checkpoint, and completion plans contain at most one call; hence body length is at most two. □

Proposition 5 (Monitor trace fidelity)**.** *For concrete PDDL execution $\xi = s_0, a_1, s_1, \ldots, a_k, s_k$, the monitor state after s_i is exactly the state obtained by running $\mathcal{D}_q$ on valuations $val_q(s_0), \ldots, val_q(s_i)$.*

Proof. The environment evaluates $val_q(s_0)$ before dispatch, obtaining $z_0 = \delta_q(q_q^0, val_q(s_0))$. Assume equality after s_{i-1}. The environment accepts a_i only when its PDDL preconditions hold, computes $s_i = \gamma(s_{i-1}, a_i)$, then replaces the monitor state by $z_i = \delta_q(z_{i-1}, val_q(s_i))$. Determinism gives the unique automaton run. Monitor symbols are query-local and cannot be effects of a PDDL action or a BDI plan, so no other transition is possible. □

Proposition 6 (Initial-acceptance trace semantics)**.** *If the automaton is accepting after the initial valuation, the committed action sequence is empty and its finite state trace is $\langle s_0 \rangle$.*

Proof. By the observation assumption, s_0 is consumed before a controller requests a primitive action. Acceptance returns immediately and commits no action. A finite trace is a nonempty sequence of states, so the corresponding trace is $\langle s_0 \rangle$. Adding a noop would append another position and is not semantically neutral for strong Next or Until. □

2.3 Numeric and Negative Obligations

Positive numeric equality is an achievement only when symbolic execution proves constant integer progress toward the target. Unit changes require strict direction; a larger constant change requires the exact predecessor. Mixed Boolean–numeric objectives require one complete final-state proof. Negated numeric equality is observation-only unless a complete action proves a change away from the forbidden value. These restrictions reduce numeric reasoning to the same completion and preservation obligations as predicates; they do not claim arbitrary numeric planning.

For numeric preparation, the certified ranking is $\rho_{\text{num}} = \langle n_{\text{miss}}, d_{\text{target}} \rangle$, where the first component counts unsatisfied producer preconditions and the second is the bounded target deficit $d_{\text{target}}(s) = |f(s) - c|$ for target equality $f = c$. For source state q_s, let $\mathcal{W}_{q_s}$ be its non-progress self-loop guards. The signed source invariants are

$$\mathcal{I}_{q_s} = \langle I^+_{q_s}, I^-_{q_s} \rangle, \qquad I^{\pm}_{q_s} = \bigcap_{\chi \in \mathcal{W}_{q_s}} P^{\pm}_{\chi}.$$

A cyclic guard may instead use b^{joint}_{χ} only when symbolic execution proves that one complete branch establishes every obligation in $\mathcal{O}_\chi$.

For strong Until, literals common to all non-progress source loops are source invariants. Every primitive prefix before progress must preserve them. This is why primitive-step monitoring and module-return monitoring are distinct experimental variants: the latter cannot justify an intermediate-state safety claim.

3 Algorithmic Correspondence

Algorithm 2 summarizes the atomic compilation pipeline whose stages are defined in the main paper and formalized above.

Algorithm 2 Compile the Certified Lifted Atomic Module Core

Require: PDDL domain D, training split $\mathcal{I}_{\text{train}}$, raw provider artifact $\mathcal{E}_{\text{raw}}$
Ensure: Certified lifted atomic module core $\mathcal{M}_D$ or a rejection diagnostic

$E_0 \leftarrow \textsc{NormalizeEvidence}(\mathcal{E}_{\text{raw}}, D, \mathcal{I}_{\text{train}})$
$E \leftarrow \textsc{CanonicalLift}(E_0, D)$
$T_D(E) \leftarrow \textsc{ProducibleTargets}(E, D)$
$\mathcal{C}_E \leftarrow \textsc{ValidateEvidenceMacros}(E, D)$
$\mathcal{C}_D(E) \leftarrow \textsc{LiftSchemas}(D, \mathfrak{G}, T_D(E), T^{\mathbb{Z}}_D(E))$
$\mathcal{C}^{\checkmark}_{D,E} \leftarrow \textsc{Certify}(\mathcal{C}_E \cup \mathcal{C}_D(E), D)$
$\mathfrak{F}_{D,E} \leftarrow \textsc{FeasibleSubsets}(\mathcal{C}^{\checkmark}_{D,E}, E)$
if $\mathfrak{F}_{D,E} = \varnothing$ **then**
  reject
end if
return $\mathcal{M}_D \leftarrow \textsc{ClingoLexMin}(\mathfrak{F}_{D,E})$

Table 4 maps each theoretical construction to the public implementation. Module docstrings repeat these references so that code and paper can be audited in both directions.

3.1 Empirical Checks of the Formal Claims

The proofs above establish the propositions for the declared formal objects. Table 5 records separate empirical checks that exercise their implementations. These checks are falsification attempts, not substitutes for the proofs: a passing finite test set cannot establish a universal theorem.

4 Data Appendix

4.1 Novel Temporal-Goal Dataset

Construction unit. For domain D and held-out problem P_i, one private construction record is

$$\mathcal{B}_i = \langle D, P_i, q_i, T_i, \theta_i, \pi_i \rangle.$$

Here q_i is the public controlled-language query, T_i the hidden lifted LTL_f oracle, θ_i its problem-object binding, and π_i a legal primitive-action witness. P_i supplies typed objects and its initial state.

The original PDDL achievement goal is retained only as provenance. It cannot affect rollout enumeration, candidate ranking, query wording, or construction success; mechanically using F of that goal would not test temporal order.

Algorithm 3 Witness-backed construction of the TEG benchmark

Require: Domain D and canonically ordered held-out problems $P_1, \ldots, P_n$
Ensure: Public rows q_i and sealed oracles (T_i, θ_i, π_i)

Parse the typed PDDL catalogue and each problem's objects and initial state.
for each P_i in canonical path order **do**
  Enumerate bounded legal, state-changing, nonrepeating rollouts Π_i.
  Extract predicate-add, predicate-delete, and integer-change events.
  Instantiate the five profiles in Table 6.
  Reject candidates that fail legality, novelty, or trace satisfaction.
  Lift objects to typed parameters; seal each binding and witness.
  Select by profile use, semantic-signature use, quality, then structural hash.
  Render q_i deterministically; emit separate public and sealed records.
end for
Deduplicate only byte-identical complete model inputs with agreeing semantics.

Legal rollouts and events. No classical or generalized planner is called. Applicable actions satisfy all positive, negative, numeric, and type conditions; effects are applied simultaneously.

Only changed successor states are retained, and no path may repeat a complete state.

A positive event changes a predicate atom from false to true; a negative event changes it from true to false. A changed integer fluent contributes equality to its successor value.

Table 6 gives the exact candidate grammar.

Formal construction	Principal public source	Verification
Evidence normalization and canonical lifting	`src/domain_level_planning/evidence_module.py`	Provider-name invariance, repeated-variable sharing, domain-constant preservation, schema, arity, numeric, and macro tests
Certified candidate generation and feasible-core selection	`src/domain_level_planning/atomic_module_synthesis.py`	Clingo coverage, refinement, internal-call closure, ranking, release, and renaming tests
Conditional module-completion summaries and threat relation	`src/domain_level_planning/certified_effects.py`	Typed unification, net effects, negative preservation, and portfolio tests
DFA progress composition	`src/domain_level_planning/temporal_goal_appender.py`	Conjunction, cyclic rejection, Until, negative, mixed numeric, and alpha-renaming tests
Balanced binary transition-repair tree	`src/domain_level_planning/transition_repair_tree.py`	Plan count, depth, fan-out, order, negative leaf, and checkpoint tests
Primitive-step execution and trace commit	`src/evaluation/jason_runtime/runner.py`	Jason execution, PDDL successor, numeric update, DFA monitor, trace, and VAL tests

Table 4: Correspondence between formal GP2PL constructions and their public realizations.

Formal claim	Empirical falsification target	Released evidence
Proposition 1 and Lemmas 1–5	Reject unbound actions, cyclic producer dependencies, non-decreasing recursion, and ambiguous resource discharge; execute accepted branches under PDDL semantics	Binding, preparation, progress, and resource challenge cases; 1,228 Jason/VAL-valid traces
Proposition 2 and Lemma 6	Reject uncovered evidence or schema-achievement contracts, open internal calls, and non-optimal encoded branch subsets	Clingo coverage, internal-call closure, refinement, and objective tests; exact selected libraries for all 16 domains
Proposition 3 and Lemmas 5–6	Reject cyclic completion threats and forbidden negative completion additions; preserve accepted signed guards	Threat and negative-guard challenge cases; conjunction, negation, Until, and mixed-numeric executions
Proposition 4	Check exact plan count, fan-out, depth, and in-order leaf visitation over singleton and conjunctive guards	Balanced transition-repair complexity and generated-controller tests
Proposition 5	Compare primitive-step monitor updates with an independent replay and direct finite-trace semantics	1,228 dual-DFA trace checks and 14 labelled semantic-conformance traces
Proposition 6	Accept an initially satisfied predicate or integer equality without inventing a primitive action	Two zero-action Jason and singleton-state replay cases

Table 5: Finite empirical falsification checks associated with the formal claims. A passing check exercises the implementation but does not replace the corresponding proof.

Profile	Hidden lifted oracle	Required grounded witness evidence
Same-state conjunction	$F(A \wedge B)$	One legal action makes distinct positive events A and B true in the same successor state.
Same-state with negation	$F(A \wedge \neg B)$	One legal action makes A true and deletes predicate atom B in the same successor state.
Ordered two milestones	$F(A \wedge X(F(B)))$	Two legal state-changing actions; A is new after the first and B is new after the second.
Ordered three milestones	$F(A \wedge X(F(B \wedge X(F(C)))))$	Three legal state-changing actions produce four pairwise-distinct states and a new selected event at each step.
Persistence Until	$A\,U\,B$	Dynamic predicate A holds in every state before a new B event, and B is absent initially.

Table 6: Version-1 temporal profiles. F is eventuality, X is strong next, and U is strong Until. The witness is grounded before A, B, and C are lifted.

Registered bounds and nontriviality. Table 7 fixes the finite search. The expanded tier is tried once and only when a problem has no primary candidate; every other rule remains unchanged.

Bound	Primary	Expanded
Trace depth	3	3
Applicable actions retained per state	12	32
Join bindings retained per schema	64	2,048
First-action prefixes	4	4
Second actions per prefix	4	4
Three-step prefixes	1	1
Third actions per prefix	4	4
Candidates per problem	1,024	1,024
Candidates per profile	16^a	16^a

Table 7: Registered construction bounds. [a]The ordered-three profile is capped at eight candidates.

Candidates are rejected when an action has no state effect, a complete state repeats, a second action restores the initial state, an ordered milestone is not new at its selected step, or an Until right-hand milestone holds initially.

Failure means no witness was found under these bounds, not that no TEG exists.

Lifting and deterministic selection. Objects are traversed in formula order. Domain constants remain constants; other first occurrences become $X, Y, Z, \ldots$ with their PDDL types.

Repeat occurrences reuse the same parameter. Type-compatible distinct objects receive explicit inequality constraints.

The ground assignment is stored only in θ_i. Each candidate receives an alpha-normalized semantic signature over its profile, types, sharing, constraints, atoms, and formula tree; ground object and witness-action names are excluded.

Within each domain, selection minimizes prior profile use, then prior signature use, a registered structural quality key, and a spelling-independent SHA-256 tie-break.

Thus an alphabetically early action schema cannot define the benchmark by being encountered first.

Concrete example. A legal witness whose milestones are `holding(b1)` and later `on(b1,b4)` is lifted with $\theta_i = \{X \mapsto b1, Y \mapsto b4\}$ to the hidden oracle $T_i = F(holding(X) \wedge X(F(on(X,Y))))$.

The public query exposes only typed X, Y, $X \neq Y$, and their strict milestone order.

Rendering, deduplication, and audit boundary. A deterministic visitor renders controlled English directly from T_i; no language model or domain-specific gloss participates.

The public manifest keeps one row per problem, while the sealed audit keeps T_i, θ_i, π_i, and replay-state fingerprints.

The 1,228 public rows are grouped into 475 translation calls by hashing the rendered domain prompt, source text, typed parameters, constraints, and parameter semantics.

Equal English alone is insufficient, and a group is rejected if its hidden semantic signatures disagree.

The implementation is in `src/temporal_input/nl_benchmark.py` and `src/temporal_input/translation_worklist.py`; construction-profile, lifting, symbol-invariance, and deduplication tests accompany both modules.

The novel GP2PL data release contains 475 deduplicated controlled-language specifications and 1,228 bound query cases over 16 PDDL domains. Each record contains the source utterance, typed parameters, binding, atom table, gold parametric LTLf formula, model-produced JSON, language-equivalence result, and sealed construction-witness checks. Per-domain JSON files, the complete benchmark, model predictions, independent validation, source archives, and SHA-256 manifests are included in the code-and-data appendix under `paper_artifacts/temporal_goal_benchmark/v1`. This dataset evaluates translation equivalence and witness validity; the 1,228 Jason, VAL, and trace-automaton executions are reported in the separate evaluation release. The released predictions use GPT-5.5 (OpenAI 2026); the model identifier and complete request/response archives are retained.

The dataset includes its own data dictionary, citation metadata, and integrity checker. Publication replaces three machine-local directory strings in the model-delivery archive with release-relative paths. The release records the original archive digest and every normalized member; model responses and semantic-validation records are unchanged.

Original natural-language annotations, temporal specifications, model outputs, generated plan libraries, and result records are released under CC BY 4.0. The GP2PL source is released under Apache-2.0. Third-party PDDL files retain their upstream terms and are reconstructed by pinned download scripts rather than relicensed as original GP2PL data.

4.2 Frozen Natural-Language Translation Prompt

The natural-language translation protocol is an evaluated input condition, separate from the policy-lifting method. Its role is to map one controlled-language specification to typed, externally bound parametric LTL_f JSON. It cannot choose PDDL actions, objects, bindings, or domain-library modules. The full protocol comprises five declared components: a normal-form guide, schematic few-shot examples, lifted-variable rules, an operator whitelist, and model-correctable retry guidance. The PDDL catalogue and public sample row are deterministic substitutions. Hidden profile labels, object bindings, gold formulas, and construction witnesses are not model-visible.

The following is the semantically complete paper-facing template. Line wrapping is typographic, and angle-bracket fields denote deterministic catalogue or sample substitutions. The release retains the exact rendered requests, responses, model identifier, prompt-source revision, and validation records.

```
SYSTEM
You translate one controlled
natural-language temporal goal into lifted
Linear Temporal Logic
```

```
on finite traces (LTLf).
Planning domain: <domain>
Your only task is semantic translation.
Do not plan or execute actions. Do not
ground lifted
parameters or choose problem objects.

PUBLIC PDDL CATALOGUE
Predicates:
  <name(argument types), one per line>
Numeric functions:
  <name(argument types), one per line>
Domain constants:
  <constant : type, one per line>
Type hierarchy (child <: parent):
  <child <: parent, one per line>

ATOM TABLE CONTRACT
- The formula uses only a0, a1, a2, ... .
- Define each symbol exactly once in atoms,
  in
  first-formula-occurrence order. Define no
  unused atom.
- Predicate entry keys: symbol, kind,
  predicate, args.
- Numeric-equality entry keys: symbol,
  kind, function, args, value. Value is an
  explicitly
  requested integer.
- Use only listed PDDL symbols, constants,
  parameter names, and arities.

LIFTED PARAMETER CONTRACT
- parameter_semantics externally_bound
  means a parameterized formula schema.
  Parameters
  are neither grounded objects nor LTLf
  quantifiers; an invocation supplies a
  type-safe assignment.
- Copy declared_parameters and constraints
  without changing their meaning. Preserve
  exact,
  case-sensitive parameter names.
- Every non-constant argument is one
  declared parameter. Do not invent, omit,
  rename, merge,
  or ground parameters.
- Check PDDL argument types and subtypes.
- Constraints restrict later assignments.
  Copy
  them as metadata, not LTLf propositions.

BENCHMARK-V1 OPERATOR CONTRACT
- Allowed operators are exactly
  F, X, U, &, !.
- F is eventuality; X is immediate next
  state; U is strong until and requires its
  right side.
- Parenthesize grouping explicitly.
- Do not use |, G, R, WX, implication,
  equivalence, quantifiers, or other
  operators.

BENCHMARK-V1 TEMPORAL STRUCTURES
- Same-state conjunction: F(a0 & a1).
- Positive plus negative: F(a0 & !a1).
- Ordered two: F(a0 & X(F(a1))).
- Ordered three:
  F(a0 & X(F(a1 & X(F(a2))))).
- Persistence through the first milestone:
  a0 U a1.
- "Strictly later" requires X; do not
  weaken it
  to F(a0 & F(a1)).

SCHEMATIC EXAMPLES
- "At some state, both conditions" ->
  F(a0 & a1).
- "First, then strictly later second" ->
  F(a0 & X(F(a1))).
- "First continues until second" ->
  a0 U a1.
The atom table supplies exact
current-domain
PDDL symbols.

OUTPUT CONTRACT
Return JSON only with exactly these eight
keys:
schema_version, sample_id, temporal_logic,
ltlf_formula, atoms, declared_parameters,
constraints, status.
Use schema_version 1, temporal_logic LTLf,
and
status supported.
Predicate atom:
{"symbol":"a0", "kind":"predicate",
 "predicate":"<listed name>",
 "args":["<parameter or constant>"]}
Numeric atom:
{"symbol":"a0", "kind":"numeric_equality",
 "function":"<listed name>",
 "args":["<parameter or constant>"],
 "value":1}
Copy sample_id, declared_parameters, and
constraints from the user message. Choose
only ltlf_formula and its atoms table. Add
no prose, markdown, explanations, object
names, or keys.

RETRY CONTRACT
A later user message may supply the
previous formula and one model-correctable
schema, syntax, vocabulary, arity, type,
parameter, operator, atom-table, or
semantic error. Correct only that
translation and
return the same eight-key JSON.
Infrastructure failures are not
instructions.

USER
Translate this public benchmark row. The
source text is the sole source of temporal
structure;
omitted metadata must not be inferred.
{
  "sample_id": "<sample identifier>",
  "source_text":
    "<controlled-language goal>",
```

```
  "declared_parameters":
    <typed parameters>,
  "constraints": <binding constraints>,
  "parameter_semantics": "externally_bound"
}
Return the required eight-key JSON object
now.

RETRY USER MESSAGE, ONLY AFTER A
CORRECTABLE ERROR
The previous translation failed a model-
correctable validation check:
{
  "previous_ltlf": "<previous formula>",
  "error_type": "<validator category>",
  "error_detail": "<precise diagnostic>",
  "hint": "<allowed correction>",
  "attempt": <positive integer>
}
Preserve the source query, parameters, and
constraints. Return the same eight-key JSON
only.
```

Prompt acceptance is not the semantic oracle. The model output first passes the exact eight-key schema, atom-table totality and one-to-one mapping, PDDL vocabulary, arity, type, parameter, and operator checks. It is then converted to a deterministic automaton by LTLf2DFA and MONA and required to be finite-trace language-equivalent to the hidden gold formula. Retries receive only model-correctable diagnostics; infrastructure failures never alter the requested semantics. This separation lets the experiment evaluate prompt-based translation without trusting a well-formed string as a correct temporal specification.

4.3 PDDL Sources and Splits

The choice of domains tests three structural sources of obligations: classical singleton-goal evidence, bounded integer effects, and feature-definable serialized subgoals. These are reporting groups, not domain routing classes. All compiler decisions are derived from schema symbols, types, preconditions, effects, evidence obligations, and selected module-completion summaries.

5 Computational Reproducibility

5.1 Hardware and Software

Experiments were conducted on an Apple M4 system with ten CPU cores and 24 GB unified memory, running macOS 26.4.1 on arm64. No GPU was used. The fixed software environment is Python 3.12.7, uv 0.8.19, Clingo 5.8.0 (Gebser et al. 2019), Tarski 0.9.1 (Ramírez and Francès 2020), LTLf2DFA 1.0.2 (Fuggitti 2020), MONA 1.4-18 (Henriksen et al. 1995), Jason 3.1.2 (Bordini, Hübner, and Wooldridge 2007), OpenJDK 24, Maven 3.9.11, Docker 28.5.1, and VAL revision `3c7a1f3` (Howey, Long, and Fox 2004); the complete revision is fixed in the release manifest. The Python dependency lockfile, external-revision setup scripts, and source commit accompany the artifact.

The external MOOSE comparison has two predeclared provenance scopes. For the twelve original MOOSE domains, we copy only the five-model mean planning coverage from Table 4 of arXiv `2511.11095v1` (Chen et al. 2025): 1079.6 of 1080 cases. For the four GP2PL-added domains, we execute the exact seed-specific Raw MOOSE models locally over five seeds and validate every nonempty returned plan with VAL. The tracked reference artifact fixes the source version, twelve per-domain means, and SHA-256 contracts for the 1080-case published scope, 148-case local extension, and 1228-case union. The scopes are never selected per domain after execution. Published MOOSE time and memory are not combined with locally measured penalized average runtime, where a failure is charged twice the cutoff (PAR-2).

The local coverage record is frozen at `paper_artifacts/gp2pl_evaluation/v1/raw_moose_extension_five_seed_summary.json`. It retains 740 portable case records, the five source-summary hashes, the five evidence-batch hashes, model/domain/problem hashes, and the common MOOSE artifact and image identifiers. Machine-local paths and commands are excluded. The seed-level valid-plan counts are 26, 23, 23, 25, and 20, giving 117 successes, 619 planner failures, and four timeouts over 740 evaluations. Per-domain five-seed means are 7.0/30 for Blocks Clear, 2.0/30 for Blocks On, 6.0/77 for Blocks Tower, and 8.4/11 for Depots. All five summaries completed with zero infrastructure failures. Their recorded Git states are retained as provenance rather than being relabelled clean; coverage admissibility instead depends on the immutable case contract, evidence and model hashes, MOOSE toolchain identity, resource limits, and VAL outcome. Runtime is not used because the registered coverage protocol permits differing worker isolation and forbids cross-hardware timing comparison.

The remaining task-level references are frozen in `external_reference_results.json` in the same release. The achievement matrix contains 844/1228 validated outcomes: 591/868 for LAMA on classical PDDL and 253/360 for ENHSP MRP+HJ on numeric PDDL. The direct temporal matrix retains all 1228 identifiers, of which 492 are supported Boolean inputs and 298 produce independently validated accepting traces; 736 remain explicitly unsupported.

The achievement-reference taxonomy is: LAMA has 591 valid plans, 249 planner failures, 24 timeouts, and four `no_plan` records; MRP+HJ has 253 valid plans, 23 planner failures, and 84 timeouts. The four LAMA `no_plan` cases have goals true in the initial state, but the registered nonempty-plan wrapper does not credit a zero-action solution. The reported 591/868 row is therefore conservative by four cases.

Within the 492 inputs accepted by the direct temporal adapter, FOND4LTLf with LAMA has 298 valid traces, 85 planner failures, and 109 compiler failures. The 736 unsupported inputs comprise 360 numeric problems and 376 identifier- encoding rejections. The latter are an adapter boundary, not a limitation of the LTL_f semantics, so the row is interpreted only on its declared supported subset.

Both primary matrices used 20 case workers. The exact 9 achievement and 46 direct-temporal infrastructure failures were rerun with one worker and merged only after input

Source	Domains	Split rule	Redistribution boundary
MOOSE companion dataset, revision `e0097051` (Chen 2026)	Eight classical and four numeric domains	Upstream `training/` and `testing/` directories	Publicly downloadable; no upstream license file detected, so fetched in place
Learner-policies repository, revision `9991926f` (Bonet 2026)	Blocksworld Clear and On	Published no-constants training and testing directories	Publicly downloadable; no upstream license file detected, so fetched in place
PDDL Instances, revision `cf19edf7` (Bacchus 2001; Potassco 2026)	Blocksworld Tower	First $\lfloor n/4 \rfloor$ naturally sorted instances train; remainder test	Publicly downloadable; no upstream license file detected, so fetched in place
D2L, revision `0620e169` (RLeap Project 2026)	Depots	First $\lfloor n/2 \rfloor$ naturally sorted instances train; remainder test	GPL-3.0 upstream repository; original terms retained

Table 8: Public PDDL sources. Commits and deterministic split rules are part of the release manifest.

hashes, limits, revisions, and toolchain identities matched. The local and remote systems have equivalent hardware configurations, and queue waiting is excluded from case runtime, so these repaired records remain eligible for PAR-2. The generated FOND4LTLf Python entry point and MONA libtool launcher embed absolute installation paths: the merge verifies each retry-file hash, rewrites only the recorded installation prefix, and requires the rewritten bytes to match the primary hash. No planner failure, timeout, unsupported input, compiler failure, or validation failure is eligible for replacement.

5.2 Parameters and Development Values

MOOSE defaults and synthesis budgets follow its published algorithm and evaluation protocol (Chen et al. 2026); declared deviations are shown rather than silently normalized.

All five MOOSE seeds use one internal worker and are compiled independently. Seed processes may be launched concurrently in isolated operating-system processes, but their evidence is never concatenated. The standalone five-seed Full GP2PL study uses eight validation workers, whereas every paired atomic and temporal ablation uses the registered six workers.

The fixed temporal release predates this five-seed protocol and conditions on one hash-identified seed-0 library set. Its 12-worker launch changes throughput, not query semantics, and does not support a five-seed robustness claim.

5.3 Metrics, Distribution, and Validation

The five-seed submission record is `paper_artifacts/gp2pl_evaluation/v1/five_seed_full_compiler_summary.json`. It retains all five source summary hashes, the common 1,228-case input-snapshot hash, per-case outcome patterns, and the exact compiler/runtime source-file-set hash. It also records the SHA-256 of the original five-seed runner aggregate and accepts that aggregate only after its protocol, run identifiers, per-seed domain counts, coverage values, and evidence-generation outcomes agree with the five child summaries. The five runs contain 6,059 original-goal VAL-valid successes among 6,140 repeated evaluations. There are no timeouts or nonzero exits. Of the 1,228 distinct cases, 1,180 succeed for every seed, 46 are seed-sensitive, and Depots `p12` and `p20` fail for every seed.

Table 10 preserves every raw domain–seed success count together with its cross-seed descriptive statistic. Table 11 records the fixed seed-0 library inputs used by the separate temporal evaluation; it is not a five-seed robustness result.

The main paper reports only the aggregate fixed-core temporal result and the same-scope evidence-to-library contrast. The profile decomposition, per-domain Raw MOOSE extension results, and complete scope-separated external references are retained here because they refine, rather than define, the headline claims.

Instance-planner outcomes. LAMA validates 591/868 classical instances, while MRP+HJ validates 253/360 numeric instances. Their PAR-2 values are 1178.4 and 1127.4 seconds. These rows cover disjoint fragments and are interpreted within scope rather than as a same-instance ranking.

FOND4LTLf with LAMA supports 492 of 1228 temporal queries and validates 298. The remaining 736 inputs are unsupported rather than planner failures; this total includes both numeric PDDL outside the tool's scope and identifiers rejected by the current adapter encoding. Supported-set PAR-2 is 1457.5 seconds. Raw MOOSE extension runtime remains omitted under its coverage-only protocol. The artifact records preserve the planner, compiler, timeout, and unsupported outcome taxonomy.

The localized variation follows the method boundary. A provider macro is a complete lifted action sequence in MOOSE's readable evidence; a Logistics example transports one package by truck, airplane, then truck. Seeded singleton-goal order changes the states from which these long macros are regressed. When one is absent, action-schema-derived candidate generation cannot always replace it because `at` may require `in` while loading to establish `in` requires `at`; this is a cyclic producer dependency with no current decreasing mode certificate. Depots `p12` and `p20` expose a different limit: a crate may be suspended while the controller must choose and bring a typed support resource. The current certificate proves finite release for a capacity resource already bound in the branch, but cannot synthesize a nested support-resource selection certificate. A generic extension would need an evidence-guided decreasing mode path for Logistics and a certified nested resource choice for Depots.

Quantity	Development values	Final value	Selection criterion
MOOSE goal size	1	1	Singleton-goal evidence definition
MOOSE goal permutations	3	3	MOOSE Algorithm 1 default
MOOSE repetition seed	0–4	0,1,2,3,4	Five independent repetitions; no pooling or best-seed choice
Internal MOOSE workers	1,6,12	1	Remove concurrent access to the seeded permutation stream
MOOSE synthesis limit	30 minutes, 12 hours	12 hours	Match the MOOSE synthesis protocol rather than its test-time cap
MOOSE memory limit	16 GiB	16 GiB	Host-safe declared deviation from MOOSE's 32 GB allowance
Action-schema search-depth bound	finite legacy probes, none	none	Termination by repeated normalized-state rejection
Compiler selector	local legacy pruning, Clingo	Clingo 5.8.0	Exact encoded constraints and lexicographic objective
Transition-repair tree branching factor	2	2	Binary structural theorem; not empirically tuned
Translation temperature	0	0	Deterministic structured translation
Translation token limit	60,000	60,000	Accommodate the complete JSON response schema
Semantic retries	0–3	at most 3	Retry only structured validation failure
Jason/VAL timeout	30 minutes	30 minutes each	Common per-instance evaluation cap
Validation workers	6,8,10,12,16,20	6 paired; 12 fixed temporal; 20 external	Fixed per registered experiment; not a method parameter
Java thread stack	default, 64 MiB	64 MiB	Prevent parser stack exhaustion on large query libraries

Table 9: Complete final parameters and values encountered during development. Method-defining constants were not selected on test performance.

Domain	Test	Seed 0	Seed 1	Seed 2	Seed 3	Seed 4	Coverage (%)
barman	90	90	90	90	90	90	100.0 ± 0.0
blocksworld-clear	30	30	30	30	30	30	100.0 ± 0.0
blocksworld-on	30	30	30	30	30	30	100.0 ± 0.0
blocksworld-tower	77	77	77	77	77	77	100.0 ± 0.0
depots	11	9	7	6	6	7	63.6 ± 11.1
ferry	90	90	90	90	90	90	100.0 ± 0.0
gripper	90	90	90	90	90	90	100.0 ± 0.0
logistics	90	88	85	54	72	90	86.4 ± 16.7
miconic	90	90	90	90	90	90	100.0 ± 0.0
numeric-ferry	90	90	90	90	90	90	100.0 ± 0.0
numeric-miconic	90	90	90	90	90	90	100.0 ± 0.0
numeric-minecraft	90	90	90	90	90	90	100.0 ± 0.0
numeric-transport	90	90	90	90	90	90	100.0 ± 0.0
rovers	90	90	90	90	90	90	100.0 ± 0.0
satellite	90	90	90	90	90	90	100.0 ± 0.0
transport	90	90	90	90	90	90	100.0 ± 0.0

Table 10: Complete Full GP2PL atomic held-out successes by independent MOOSE goal-order seed. Coverage is mean $\pm$ sample standard deviation (SD) for $n = 5$ independently compiled atomic cores; no evidence is pooled and no best seed is selected.

Domain	Tr/Te	E/S	Sel.	E2E	s
barman	30/90	176/277	441	90/90	11.3
blocksworld-clear	120/30	7/30	34	30/30	3.9
blocksworld-on	150/30	53/30	80	30/30	3.7
blocksworld-tower	25/77	179/29	205	77/77	7.1
depots	11/11	176/156	323	11/11	13.1
ferry	20/90	5/17	21	90/90	4.2
gripper	3/90	4/25	26	90/90	4.4
logistics	30/90	87/28	111	90/90	8.4
miconic	30/90	11/10	20	90/90	5.2
numeric-ferry	10/90	5/13	17	90/90	5.9
numeric-miconic	30/90	11/16	26	90/90	5.3
numeric-minecraft	2/90	17/5	22	90/90	4.3
numeric-transport	10/90	5/13	16	90/90	5.2
rovers	99/90	90/40	127	90/90	9.7
satellite	99/90	15/26	40	90/90	5.6
transport	20/90	5/15	18	90/90	4.2

Table 11: Per-domain fixed seed-0 atomic cores and temporal execution. Tr/Te denotes train/test instances; E/S denotes evidence/schema candidates; Sel. denotes selected branches; E2E requires Jason, neutral-goal VAL, and both DFA checks; s is median wall-clock seconds per query.

Profile	DFA equiv.	E2E valid	Med. act.	Med. s
Ordered-3	137/137	272/272	3	5.4
Ordered-2	136/136	273/273	2	5.6
Strong Until	119/119	275/275	1	5.4
Conjunction	19/19	137/137	1	5.4
Conj.+negation	64/64	271/271	1	5.6
All	475/475	1,228/1,228	2	5.5

Table 12: Translation and execution by temporal profile. DFA equiv. counts the 475 distinct translation inputs whose gold and predicted DFAs have exactly the same finite-trace language. E2E valid counts all 1,228 bound queries and requires controller compilation, Jason completion, neutral-goal VAL, and acceptance by both DFA trace oracles. Medians are primitive actions and wall-clock seconds; the All row is pooled across all bound queries.

Scope	Valid/seed	Coverage (%)
Reported (MOOSE Table 4)		
Original 12 domains	1079.6/1,080	99.96
Measured (local five seeds)		
GP2PL-added 4	23.4 ± 2.3/148	15.81 ± 1.56
Blocks Clear	7.0 ± 0.0/30	23.33 ± 0.00
Blocks On	2.0 ± 0.0/30	6.67 ± 0.00
Blocks Tower	6.0 ± 1.7/77	7.79 ± 2.25
Depots	8.4 ± 1.1/11	76.36 ± 10.37

Table 13: Raw MOOSE coverage under the preregistered source split. The Reported row is copied from Table 4 of the five-seed MOOSE extended paper (Chen et al. 2025); the Measured rows are mean $\pm$ sample standard deviation over the five local evidence seeds. The scopes are disjoint, and no cross-hardware runtime comparison is made.

Domain-specific transport or parking rules are not admissible substitutes.

All five seeded runs share the same sealed PDDL snapshot and hash-verified method-defining source-file set. The machine record preserves the source and protocol fingerprints. Timing is excluded from cross-method speed and significance claims because the sequential runs overlapped unrelated machine workloads. A supplementary diagnostic artifact may report the descriptive, within-method cumulative distribution of Jason subprocess `run_seconds`; it excludes queue waiting and VAL time, keeps failures in the denominator, and is not evidence of superiority over another planner.

Atomic measures are producible-predicate trigger coverage, internal-call closure, held-out Jason plus original-goal VAL success, selected branches, context and body cost, library bytes, and compilation time. Temporal success requires all of: controller acceptance, Jason completion, neutral-goal VAL acceptance, independent PDDL replay, gold-DFA acceptance, and predicted-DFA acceptance. Failed action prefixes are diagnostics and never plans.

The atomic matrix repeats each of 1,228 held-out case identifiers under five evidence seeds. Seed–case totals are descriptive; they are not treated as 6,140 independent experimental units. For the Direct Producers–Maximum Feasible contrast, we average the five paired binary differences for each case identifier and apply a two-sided exact sign test to the nonzero case-level differences.

The temporal matrix contains one query per held-out problem under one fixed atomic core. Queries nevertheless cluster by domain and may share one of 475 translated inputs. We therefore report paired discordant counts and their domain/profile concentration, but attach no case-level p-value to temporal coverage differences.

The completed paired release is `paper_artifacts/gp2pl_evaluation/v1/paired_ablation_results.json`. It contains 24,560 portable atomic outcomes (1,228 cases, four variants, and five seeds), 4,912 portable temporal outcomes (1,228 cases and four variants), every paired-input contract, the 13 challenge count, and exact parent-result and challenge-summary hashes. Machine-local paths and transient logs are excluded.

Direct Producers and Maximum Feasible have 640 seed–case outcomes favoring Maximum Feasible and none favoring Direct Producers. Aggregation within case gives 130 nonzero case-level differences, all positive; the two-sided exact sign test yields $p = 1.47 \times 10^{-39}$. Maximum Feasible and Full GP2PL have identical valid-case sets; Full removes 34 mean branches and 10.8 mean KiB. Evidence Only and Direct Producers differ on one seed–case outcome in the opposite direction, so target completion alone is not claimed to make AgentSpeak plan selection monotone.

Unprotected Serialization and Certified Flat have 115 cases favoring certification and 1 favoring the unprotected controller. The four numeric domains contribute net gains of 25, 28, 29, and 31; Transport contributes two, and Logistics contributes one loss. Thus 113 of the 114 net gains arise in the bounded-integer subset. Certified Flat and Certified Balanced have identical valid-case sets; the balanced

Method	Source	Scope	Coverage	Unsupported	PAR-2 s
MOOSE	Reported	Original MOOSE domains, five seeds	1079.6/1080	0	–
Raw MOOSE extension	Measured	Added domains, five seeds	23.4 ± 2.3/148	0	–
LAMA	Measured	Classical achievement	591/868	0	1178.4
MRP+HJ	Measured	Numeric achievement	253/360	0	1127.4
FOND4LTLf + LAMA	Measured	Supported Boolean TEG	298/492	736	1457.5

Table 14: Scope-separated external planning references. Reported MOOSE coverage is copied from Table 4 of the five-seed extended paper (Chen et al. 2025); its runtime is not compared with local measurements. Measured rows use a 1,800-second, 8-GiB per-task budget, and PAR-2 charges nonvalid supported cases twice the cutoff. Raw MOOSE extension coverage is mean ± sample standard deviation over five seeds; its runtime is omitted by protocol. FOND4LTLf unsupported inputs are excluded from its coverage denominator and separated from planner and compiler failures. Rows with different scopes are not ranked.

Method	Valid (%)	Branches	KiB
Evidence Only	88.3	835.0 ± 15.8	488.3 ± 8.3
Direct Producers	88.3	1153.0 ± 15.8	549.2 ± 8.3
Maximum Feasible	**98.7**	1533.0 ± 15.8	645.9 ± 8.3
Full GP2PL	**98.7**	**1499.0 ± 15.8**	**635.1 ± 8.3**

Table 15: Paired atomic compiler comparison over 6,140 held-out seed–case executions (16 domains, 1,228 cases, five fixed seeds). All four variants compile all 80 libraries and reach the 365 producible targets except Evidence Only (90/365). Valid requires Jason success and original-goal VAL; Branches and KiB (library size) are mean ± sample standard deviation across seeds. Bold marks tied best coverage; blue bold marks Full GP2PL and its smaller core at equal coverage. Full configuration and timing appear in the Technical Supplement.

Method	Valid	PAR-2 s	Plans	Fan-out
Unprotected Serialization	1113/1228	342.0	7	3
Certified Flat	**1227/1228**	8.0	7	3
Certified Balanced	**1227/1228**	8.6	13	**2**
Module-Return Monitor	1211/1228	56.0	13	2

Table 16: Paired temporal compiler comparison over 1,228 queries with identical DFA, binding, and atomic-library inputs; all four build controllers for every query. Valid requires Jason, neutral-goal VAL, and acceptance by the gold and predicted DFA trace oracles. PAR-2 charges failures twice the 1,800-second limit. Plans is the median controller size and Fan-out its maximum transition-repair value. Bold marks tied best coverage; blue bold marks Balanced and its lower fan-out relative to equal-coverage Flat. Color is not used alone.

result therefore supports only the structural fan-out claim. Certified Balanced and Module-Return Monitor have 16 discordant cases, all favoring primitive-step observation. PAR-2 and action counts remain descriptive; no continuous-measure superiority claim is made.

For the 1,228 fixed temporal cases, runtime has minimum 2.29, first quartile 4.28, median 5.49, third quartile 8.49, maximum 110.31, mean 9.39, and sample standard deviation 13.66 seconds. Primitive-action count has minimum 1, first quartile 1, median 2, third quartile 2, maximum 4, mean 1.75, and sample standard deviation 0.80. The complete records, rather than only these summaries, are included.

Thirteen registered challenge cases all pass. Seven exercise rejection or exact acceptance for binding, cyclic preparation, obstruction exchange, ambiguous resource capacity, cyclic transition threats, forbidden negative completion, and exact terminal numeric progress. Six test invariance under predicate/action renaming, action-parameter permutation, evidence-object renaming, irrelevant static vocabulary, negative-guard renaming, and progress-relation renaming.

The completed matrix pairs binary atomic success by seed, domain, and test, and binary temporal success by query under identical inputs. The atomic sign test uses distinct case identifiers as its inferential unit; temporal contrasts are descriptive because domain and translation-input clustering remains. The Flat versus Balanced claim is structural because their success sets are identical. Continuous values are reported on the registered denominator or jointly solved cases without a superiority test.

5.4 Reproduction Commands and Artifact Map

The code-and-data appendix contains every GP2PL-authored source file required to conduct and analyze the experiments: compiler and runtime source, tests, preprocessing and benchmark-materialization programs, experiment drivers, validation harnesses, table generators, the dependency lockfile, novel data, fixed generated libraries, compact execution records, and manifests. The same source is released under Apache-2.0. Third-party planners, interpreters, and benchmark repositories are not represented as author-created code; supplied setup scripts retrieve them at recorded revisions and preserve their upstream terms.

Generator-assisted domain onboarding. For a future domain, a third-party PDDL problem generator may supply

$\mathcal{I}_{\text{train}}$ and $\mathcal{I}_{\text{test}}$ before evidence extraction. Reproducibility requires the generator revision and digest, parameter ranges, random seeds, deterministic split rule, and complete output manifest with file digests.

Every generated problem must parse against D, training and held-out content hashes must be disjoint, and the held-out set must be sealed before evidence learning. The generalized-planning backend receives only $\mathcal{I}_{\text{train}}$; GP2PL receives its normalized evidence. A generator supplies coverage opportunities, not certificates, so every compiler gate remains unchanged.

```
uv sync
bash scripts/setup_mona.sh
bash scripts/setup_moose.sh
bash scripts/setup_benchmark_sources.sh
uv run python \
  scripts/materialize_achievement_\
benchmarks.py
uv run pytest -p no:cacheprovider -q
uv run python \
  scripts/run_certificate_challenge_\
matrix.py
```

The fixed evaluation tables are regenerated with the conference-neutral generator:

```
RELEASE=paper_artifacts/gp2pl_evaluation/v1
uv run python \
  scripts/generate_evaluation_tables.py \
  --execution-summary \
  "$RELEASE/\
temporal_execution_summary.json" \
  --benchmark-compatibility \
  "$RELEASE/benchmark_compatibility.json" \
  --atomic-library-root \
  "$RELEASE/atomic_libraries" \
  --output-dir \
  latex_code/aamas_method_paper/sections
```

The independent five-seed Full GP2PL table and compact submission record are regenerated directly from the five complete Jason/VAL summaries with:

```
AGG=artifacts/parser_order_full_val_logs/\
pddl-five-seed-20260713-153900/\
five_seed_summary.json
uv run python \
  scripts/freeze_five_seed_full_\
compiler_results.py \
  --source-aggregate "$AGG" \
  --compiler-freeze-revision 5e632fb5
```

The paired ablation release and both comparison tables are regenerated from the completed matrix without rerunning any planner:

```
uv run python \
  scripts/freeze_paired_ablation_\
results.py \
  --paired-results artifacts/\
paired_compiler_experiments/\
aaai-paired-72b0604f/paired_results.json \
  --challenge-summary artifacts/\
paired_compiler_experiments/\
aaai-paired-72b0604f/challenge_runs/\
aaai-paired-72b0604f-certificate-\
challenges/\
summary.json
```

The compatibility certificate permits only explicitly named fields below the benchmark's provenance object. It reconstructs and verifies the exact benchmark hash recorded by the execution summary; domain cases, formulas, bindings, and expected semantics cannot be changed by this mechanism.

The former three-panel empirical view remains reproducible as a supplementary diagnostic artifact generated from the frozen paired-ablation release rather than from manually copied table values. It is not included in the main paper; the exact endpoint results appear in the generated tables. The release itself is derived from the complete registered runner matrix by the preceding command:

```
RESULT=paper_artifacts/gp2pl_evaluation/\
v1/paired_ablation_results.json
uv run python \
  scripts/generate_aaai_figures.py \
  --ablation-results "$RESULT" \
  --output-file \
  latex_code/aamas_method_paper/\
figures/fig3_evaluation.png
```

The generator emits a 4,200 by 1,700 pixel PNG at 600 dpi. Panel (a) recomputes the four-variant atomic coverage matrix from all five paired seeds, displays every domain on which any variant differs, and aggregates only the 11 domains on which all variants reach 100%. Panel (b) uses all 385 seed–instance records from the registered Blocksworld Tower size series, retaining failures in each cumulative denominator. Panel (c) marks 1, 10, 100, and 1,800 seconds and retains every failed temporal query in its denominator. Generation tests reject an incomplete pairing, an inconsistent success oracle, changed dimensions, or missing provenance.

The public release supplies an Apache-2.0 code license, a CC BY 4.0 license for original GP2PL data, third-party notices, citation metadata, and a versioned release of the novel temporal-goal dataset. During double-blind review, the same files are supplied as an anonymized code-and-data appendix and the identifying web link is disabled in the main-paper source. All external benchmark inputs are publicly retrievable at the cited revisions. No dataset used by the experiments remains unavailable; the checklist's universal condition concerning unavailable datasets is therefore satisfied, rather than being used to imply that a hidden dataset exists.

6 Reproducibility Checklist

The main checklist answers "yes" when the criterion is satisfied by the main paper together with this appendix and the submitted code-and-data archive. Specifically, this appendix supplies all formal assumptions and proofs; the novel dataset description, complete released records, and CC BY 4.0 license; citations, revisions, and deterministic split rules for existing public data; all author-created preprocessing, execution, and analysis code; complete parameters; hardware and software; distributional statistics; and a theory-to-code map.

The availability answers distinguish authored artifacts from dependencies. Every GP2PL-authored source file

needed by the reported experiments is included and Apache-2.0 licensed. External tools and benchmark repositories remain third-party works: the appendix cites them, records exact revisions, and supplies deterministic retrieval scripts without asserting ownership or relicensing them. Likewise, the sealed construction audit was unavailable to the translation model during inference but is included in the released temporal-goal dataset for independent validation. Hence there is no non-public experimental dataset.

The statistical answer is grounded in the frozen paired records. Atomic coverage inference uses a two-sided exact sign test over distinct held-out case identifiers after within-case aggregation across seeds. Temporal coverage, structural equality, and continuous summaries are reported descriptively and are not relabelled as independently sampled performance improvements. Complete per-instance outcomes remain available for independent reanalysis.

References

Bacchus, F. 2001. The AIPS '00 Planning Competition. *AI Magazine*, 22(3): 47–56.

Bonet, B. 2026. Learner Policies from Examples: Benchmark Repository. GitHub repository. Revision 9991926f7655c4b6c8dc2f0404123639e42056f2.

Bordini, R. H.; Hübner, J. F.; and Wooldridge, M. 2007. *Programming Multi-Agent Systems in AgentSpeak Using Jason*. John Wiley & Sons.

Chen, D. Z. 2026. MOOSE Companion Benchmark Dataset. GitHub repository. Revision e00970516154e9042b783a4613a1ed7286c9beee.

Chen, D. Z.; Hofmann, T.; Klassen, T. Q.; and McIlraith, S. A. 2025. Satisficing and Optimal Generalised Planning via Goal Regression. Version 1, arXiv:2511.11095.

Chen, D. Z.; Hofmann, T.; Klassen, T. Q.; and McIlraith, S. A. 2026. Satisficing and Optimal Generalised Planning via Goal Regression. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, 36197–36206. Palo Alto, California: AAAI Press.

De Giacomo, G.; and Vardi, M. Y. 2013. Linear Temporal Logic and Linear Dynamic Logic on Finite Traces. In *Proceedings of the Twenty-Third International Joint Conference on Artificial Intelligence*, 854–860. AAAI Press.

Fikes, R. E.; Hart, P. E.; and Nilsson, N. J. 1972. Learning and Executing Generalized Robot Plans. *Artificial Intelligence*, 3: 251–288.

Fox, M.; and Long, D. 2003. PDDL2.1: An Extension to PDDL for Expressing Temporal Planning Domains. *Journal of Artificial Intelligence Research*, 20: 61–124.

Fuggitti, F. 2020. LTLf2DFA: Translate LTLf and PLTLf Formulae to Deterministic Finite Automata.

Gebser, M.; Kaminski, R.; Kaufmann, B.; and Schaub, T. 2019. Multi-shot ASP Solving with Clingo. *Theory and Practice of Logic Programming*, 19(1): 27–82.

Henriksen, J. G.; Jensen, O. J. L.; Jørgensen, M. E.; Klarlund, N.; Paige, R.; Rauhe, T.; and Sandholm, A. B. 1995. MONA: Monadic Second-Order Logic in Practice. *BRICS Report Series*, 2(21).

Howey, R.; Long, D.; and Fox, M. 2004. VAL: Automatic Plan Validation, Continuous Effects and Mixed Initiative Planning Using PDDL. In *Proceedings of the 16th IEEE International Conference on Tools with Artificial Intelligence*, 294–301. IEEE Computer Society.

McDermott, D.; Ghallab, M.; Howe, A.; Knoblock, C.; Ram, A.; Veloso, M.; Weld, D.; and Wilkins, D. 1998. PDDL—The Planning Domain Definition Language.

Meneguzzi, F.; and De Silva, L. 2015. Planning in BDI Agents: A Survey of the Integration of Planning Algorithms and Agent Reasoning. *The Knowledge Engineering Review*, 30(1): 1–44.

OpenAI. 2026. GPT-5.5 System Card. OpenAI system card.

Potassco. 2026. PDDL Instances from the International Planning Competitions. GitHub repository. Revision cf19edf7c53d1540ddbb396c642595e0926ee552.

Ramírez, M.; and Francès, G. 2020. Tarski: An AI Planning Modeling Framework. Software documentation.

Rao, A. S. 1996. AgentSpeak(L): BDI Agents Speak Out in a Logical Computable Language. In *Agents Breaking Away*, Lecture Notes in Computer Science, 42–55. Berlin, Heidelberg: Springer Berlin Heidelberg.

RLeap Project. 2026. D2L: Description Logic Features for Planning. GitHub repository. Revision 0620e169c894d79b3c84f435dba1462996f7c270.

Thangarajah, J.; Padgham, L.; and Winikoff, M. 2003. Detecting and Avoiding Interference Between Goals in Intelligent Agents. In *Proceedings of the Eighteenth International Joint Conference on Artificial Intelligence*, 721–726. Morgan Kaufmann.